

Tema propusă de proiect:

Simularea unui automat celular: Furnica lui Langton în 2D (MPI)

1. Descriere detaliată (științifică) a problemei

Furnica lui Langton este un automat celular bidimensional definit de Christopher Langton în 1986, reprezentând un sistem dinamic discret cu comportament emergent complex. Spațiul de stare este o grilă infinită (în practică toroidală sau cu borduri fixe) de celule binare — albe și negre — pe care se deplasează unul sau mai mulți agenți (*furnici*).

Regula de tranziție pentru o singură furnică:

- Dacă furnica se află pe o celulă albă: rotește 90° la dreapta, inversează culoarea celulei, avansează un pas.
- Dacă furnica se află pe o celulă neagră: rotește 90° la stânga, inversează culoarea celulei, avansează un pas.

Comportamentul sistemului are trei faze distincte, studiate în literatura de specialitate:

1. Faza de simplitate (primele ~ 500 pași): tipare simetrice, aparent regulate.
2. Faza de haos (~ 500 – 10.000 pași): deplasare aparent haotică, fără structură vizibilă.
3. Faza de „highway” (după ~ 10.000 pași): furnica construiește o structură periodică diagonală repetitivă la nesfârșit (perioadă 104 pași), numită *highway* sau *corridor*.

Din perspectivă matematică, furnica lui Langton este Turing-completă și echivalentă cu o mașină Turing cu două simboluri. Comportamentul pe termen lung (demonstrarea universalității highway-ului pe o grilă infinită inițial albă) a fost conjecturat de Langton și dovedit formal abia în 2000 de Bunimovich și Troubetzkoy.

Extensia cu mai multe furnici (varianta *Turmites* sau *multi-ant*) generează dinamici calitativ diferite: furnici cu reguli identice pot genera structuri simetrice, cristale, sau comportament haotic ireductibil, în funcție de pozițiile și orientările inițiale. Această variantă face problema mai interesantă computațional, deoarece interacțiunile dintre furnici introduc dependențe de date nelocale — două furnici pot modifica aceeași celulă în același pas, necesitând o strategie de rezolvare a conflictelor.

Relevanța paralelizării: Pe grile mari (10.000×10.000 celule, milioane de pași, mii de agenți), calculul secvențial devine prohibitiv. Partiționarea domeniului pe procese MPI reduce dramatic timpul de simulare, cu prețul introducerii comunicării la frontieră și al gestionării migrației agenților între domenii.

2. Descriere succintă a problemei (ca algoritm)

```
INPUT:  Grilă  $G[N \times N]$ , listă de agenți  $A = \{(x_i, y_i, dir_i)\}$ ,  
        număr pași  $T$ , număr procese  $P$   
PARTIȚIONARE: Fiecare proces  $p$  primește rândurile  $[p \cdot N/P, (p+1) \cdot N/P)$   
               → subgrilă locală + 2 rânduri ghost (sus/jos)  
PENTRU fiecare pas  $t = 1..T$ :  
    1. COMUNICARE GHOST ROWS  
        - Trimite primul rând local → procesul  $p-1$  (ghost de jos)  
        - Trimite ultimul rând local → procesul  $p+1$  (ghost de sus)  
        - Recepționează ghost rows cu MPI_Sendrecv  
    2. ACTUALIZARE AGENȚI (locali + în ghost zone)  
        PENTRU fiecare furnică  $a_i$  pe domeniul local:  
            - Citește culoarea celulei curente  $G[x_i][y_i]$   
            - Actualizează direcția (rotație stânga/dreapta)  
            - Inversează culoarea celulei  
            - Avansează un pas →  $(x_i', y_i')$   
            - DACĂ  $(x_i', y_i')$  iese din domeniu local:  
                → Adaugă furnica în coada de migrație  
    3. MIGRARE AGENȚI  
        - Trimite furnicile din coadă proceselor vecine (MPI_Send)  
        - Recepționează furnicile sosite din vecini (MPI_Recv)  
        - Integrează furnicile sosite în lista locală  
    4. REZOLVAREA CONFLICTELOR  
        - Dacă două furnici ajung pe aceeași celulă în același pas:  
            → Prioritate prin ID (sau anulare reciprocă, configurat)  
    5. COLECTARE PERIODICĂ (la fiecare  $K$  pași)  
        - MPI_Gather → procesul root reconstruiește grila completă  
        - Export PPM/PNG pentru vizualizare / animație  
    6. CONDIȚIE DE OPRIRE  
        - MPI_Allreduce pentru verificare globală (detectare highway,  
          număr maxim de pași atins)  
OUTPUT: Secvență de imagini (animație), statistici de convergență
```

3. Modalitatea de dezvoltare — plan realist pe 4 săptămâni

Săptămâna 1 — Infrastructură și versiunea secvențială

Zi	Activitate
1–2	Setup mediu: cluster MPI (OpenMPI/MPICH), CMake, Git. Citire bibliografie (Langton 1986, Bunimovich 2000).
3–4	Implementare secvențială completă: grilă 2D, o furnică, T pași, export PPM. Validare vizuală (verificarea apariției highway-ului după ~10.000 pași).
5–7	Extindere secvențială: suport multi-furnică, rezolvare conflicte, parametrizare CLI (-n, -t, -ants).

Livrabil: executabil secvențial validat, test pe grilă 1000×1000, 100.000 pași.

Săptămâna 2 — Partiționare MPI și comunicare ghost rows

Zi	Activitate
1–2	Partiționare domeniului: calcul offset-uri, MPI_Scatter pentru distribuirea grilei inițiale.
3–4	Implementare comunicare ghost rows cu MPI_Sendrecv (tabel dublu-buffer pentru consistență).
5–6	Testare corectitudine: comparare rezultat MPI (1 proc) vs. secvențial. Unit tests per rând.
7	Refactorizare: separare clar a funcțiilor compute_step(), exchange_ghosts(), collect_grid().

Livrabil: versiune MPI funcțională fără migrarea agenților (furnici staționare sau constrânse la domeniu fix).

Săptămâna 3 — Migrarea agenților și optimizări

Zi	Activitate
1–3	Implementare protocol de migrație: coadă de ieșire per proces, MPI_Send/Recv asincron (MPI_Isend/Irecv), recepție cu MPI_Probe pentru mesaje de dimensiune variabilă.
4–5	Testare cu scenarii de stres: mii de furnici care traversează frontierele frecvent. Depanare race conditions.

Zi	Activitate
6-7	Colectare periodică (MPI_Gather) și generare animație; optimizare frecvență de colectare (nu la fiecare pas).

Livrabil: versiune MPI completă, testată pe 2, 4, 8 procese, animație corectă.

Săptămâna 4 — Benchmarking, raport și prezentare

Zi	Activitate
1-2	Campanie de benchmarking: grile 500^2 , 1000^2 , 5000^2 , procese 1/2/4/8/16, calcul speedup și eficiență.
3-4	Redactare referat (8-12 pag), grafice speedup, concluzii.
5-6	Pregătire demo live: rulare pe cluster, vizualizare animație, prezentare slide-uri.
7	Buffer: rezolvare probleme de ultim moment, revizuire cod, comentarii.

4. Evaluarea performanțelor obținute

Metrici principale

Speedup (legea Amdahl): $S(p) = \frac{T_1}{T_p}$

Eficiență: $E(p) = S(p) \times \frac{1}{p}$

Fracția serială estimată (din datele experimentale): $f = 1 - \frac{1}{S(p)}$

Experimente recomandate

Experiment	Parametri variați	Ce se măsoară
Strong scaling	$P = 1, 2, 4, 8, 16$; $N=5000$ fix	Speedup, eficiență
Weak scaling	$N^2/P = \text{const}$ (ex: $500^2/\text{proc}$)	Timp aproximativ constant
Overhead migrație	Densitate furnici: 1, 10, 100, 1000	Timp comunicare vs. calcul
Frecvența colectării	Gather la fiecare 1, 10, 100, 1000 pași	Impact I/O asupra timpului total

Rezultate așteptate și interpretare

- Strong scaling: eficiență bună (>70%) până la 8 procese; degradare la 16+ din cauza overhead-ului de migrație și a raportului comunicare/calcul.
- Weak scaling: timp aproape constant dacă densitatea de furnici/domeniu rămâne fixă — validează scalabilitatea arhitecturii.
- Bottleneck identificat: MPI_Gather periodic este cel mai costisitor; reducerea frecvenței de la 1 la 100 pași poate reduce timpul total cu 30–50%.

Instrumente de profiling recomandate

- MPI_Wtime() pentru timere granulare (calcul vs. comunicare vs. I/O)
- TAU Performance System sau Score-P pentru profil detaliat MPI
- Vampir / Jumpshot pentru vizualizarea traseelor de comunicare

5. Capitolele obligatorii ale părții teoretice (referat 8–12 pagini)

1. Introducere (~0,5 pag) Motivație, obiective, structura referatului.
2. Automate celulare — fundamente teoretice (~1 pag) Definiție formală (grilă, stări, vecinătate, funcție de tranziție). Clasificarea lui Wolfram (clase I–IV). Poziționarea Furnicii Langton în clasa IV.
3. Furnica lui Langton — descriere și proprietăți (~1,5 pag) Regula de tranziție formalizată. Cele trei faze comportamentale (simplitate, haos, highway). Teorema Bunimovich–Troubetzkoy (2000) — enunț și implicații. Extensii: Turmites, reguli generalizate (LRRL etc.), multi-agent.
4. Complexitatea computațională și necesitatea paralelizării (~0,5 pag) Analiza complexității $O(N^2 \cdot T)$. Estimări timp secvențial pentru grile mari. Argumente pentru paralelizare.
5. Modele de programare paralelă — MPI (~1 pag) Modelul SPMD. Topologii de comunicare. Operații colective vs. punct-la-punct. Sincronizare și consistență.
6. Strategia de partiționare a domeniului (~1 pag) Partiționare 1D (pe rânduri) vs. 2D (bloc). Alegere motivată. Calculul ghost layers. Diagrama domeniilor.
7. Protocolul de comunicare și migrarea agenților (~1,5 pag) Ghost row exchange cu MPI_Sendrecv. Protocolul de migrație: serializare agent, MPI_Isend/Irecv, MPI_Probe. Rezolvarea conflictelor de scriere (două furnici pe aceeași celulă).
8. Implementare — detalii tehnice (~1 pag) Structuri de date, schema de buffering (double buffer), parametrizare, generare output PPM.

9. Rezultate experimentale și analiza performanțelor (~1,5 pag) Tabele și grafice speedup/eficiență. Strong vs. weak scaling. Identificarea bottleneck-urilor. Comparatie cu predicțiile legii Amdahl.
10. Concluzii și direcții viitoare (~0,5 pag)
11. Bibliografie (~0,5 pag) Minim 8 surse: Langton (1986), Bunimovich & Troubetzkoy (2000), Gropp et al. — *Using MPI*, manuale OpenMPI, articole de benchmark MPI.

6. Concluzii finale

Furnica lui Langton reprezintă o alegere excelentă pentru un proiect de algoritmi paraleli deoarece combină simplitatea regulii locale cu complexitatea comportamentului emergent global — tocmai tipul de problemă pe care paralelizarea îl poate accelera semnificativ fără a-i altera semantica.

Principalele provocări tehnice — comunicarea ghost rows, migrarea dinamică a agenților între procese și rezolvarea conflictelor — acoperă direct conceptele fundamentale ale programării distribuite cu MPI: comunicare punct-la-punct sincronă și asincronă, mesaje de dimensiune variabilă, operații colective și echilibrul calcul/comunicare.

Din perspectiva rezultatelor așteptate, proiectul demonstrează că partiționarea de domeniu funcționează bine când domeniul este uniform și furnicile sunt distribuite aleatoriu; performanța degradează previzibil când furnicile se concentrează la frontiere (migrații frecvente) sau când colectarea pentru vizualizare este prea deasă. Aceste observații conduc natural la soluții de optimizare (reducerea frecvenței de gather, buffering asincron) care aprofundează înțelegerea compromisurilor din computing-ul paralel.

Bibliografie

Cărți și monografii

1. Langton, C. G. (1986). *Studying artificial life with cellular automata*. Physica D: Nonlinear Phenomena, 22(1–3), 120–149.
2. Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media, Champaign, Illinois. ISBN: 978-1-57955-008-0.

Articole științifice

3. Bunimovich, L. A., & Troubetzkoy, S. E. (1992). *Recurrence properties of Lorentz lattice gas cellular automata*. Journal of Statistical Physics, 67(1–2), 289–302. [doi.org](https://doi.org/10.1007/BF02186781)
4. Gajardo, A., Moreira, A., & Goles, E. (2002). *Complexity of Langton's ant*. Discrete Applied Mathematics, 117(1–3), 41–50. [doi.org](https://doi.org/10.1016/S0167-5060(01)00101-1)

Resurse tehnice MPI și calcul paralel

5. Message Passing Interface Forum. (2021). *MPI: A Message-Passing Interface Standard, Version 4.0*. Disponibil la: [mpi-forum.org](https://www.mpi-forum.org/)
6. Open MPI Development Team. (2024). *Open MPI Documentation, Version 5.0*. Disponibil la: [open-mpi.org](https://www.open-mpi.org/)

Resurse web și documentație suplimentară

7. Weisstein, E. W. *Langton's Ant*. MathWorld — A Wolfram Web Resource. Disponibil la: [mathworld.wolfram.com](https://mathworld.wolfram.com/LangtonsAnt.html)
8. Johnston, H. (2012). *Cellular Automata and Langton's Ant — Lecture Notes*. University of Massachusetts. Disponibil la: [cs.umass.edu](https://cs.umass.edu/~johnston/)